

DataFun · Agentic AI 大会

从写代码， 到构建产品

AI Agent 正在改变 Builder 的工作流

SPEAKER

Mai Yang · 杨文 · Cursor Ambassador



Mai Yang · 杨文 · Shenzhen

一个产品，三份一手记录

3天做出 MVP， 76天走到 App Store

01 真实构建

04-14 开始，04-16 交付 0.1.0；第 3 天已经是一个可以每天使用的 App。

02 完整留档

468 commits、213 段 AI 对话、11 条 rules——每次判断和翻车都有记录。

03 工程视角

十余年后端工程师——正因为会写代码，才更清楚这次变化改变的不是「写代码」本身。

先说一件反常识的事

一个用 AI 造出来的 App

但是

它现在，没有任何 AI 功能。

从 MVP 到上架的流水账

先看数字

从开工到上架

~~76天~~ → **3天**

04-14 写下第一行，04-16 就能每天用；一路打磨到 06-29 提交 App Store，正好 76 天。

协作密度

468/213

commits / AI 对话段数，每天 2.6 段对话。

给 AI 的制度

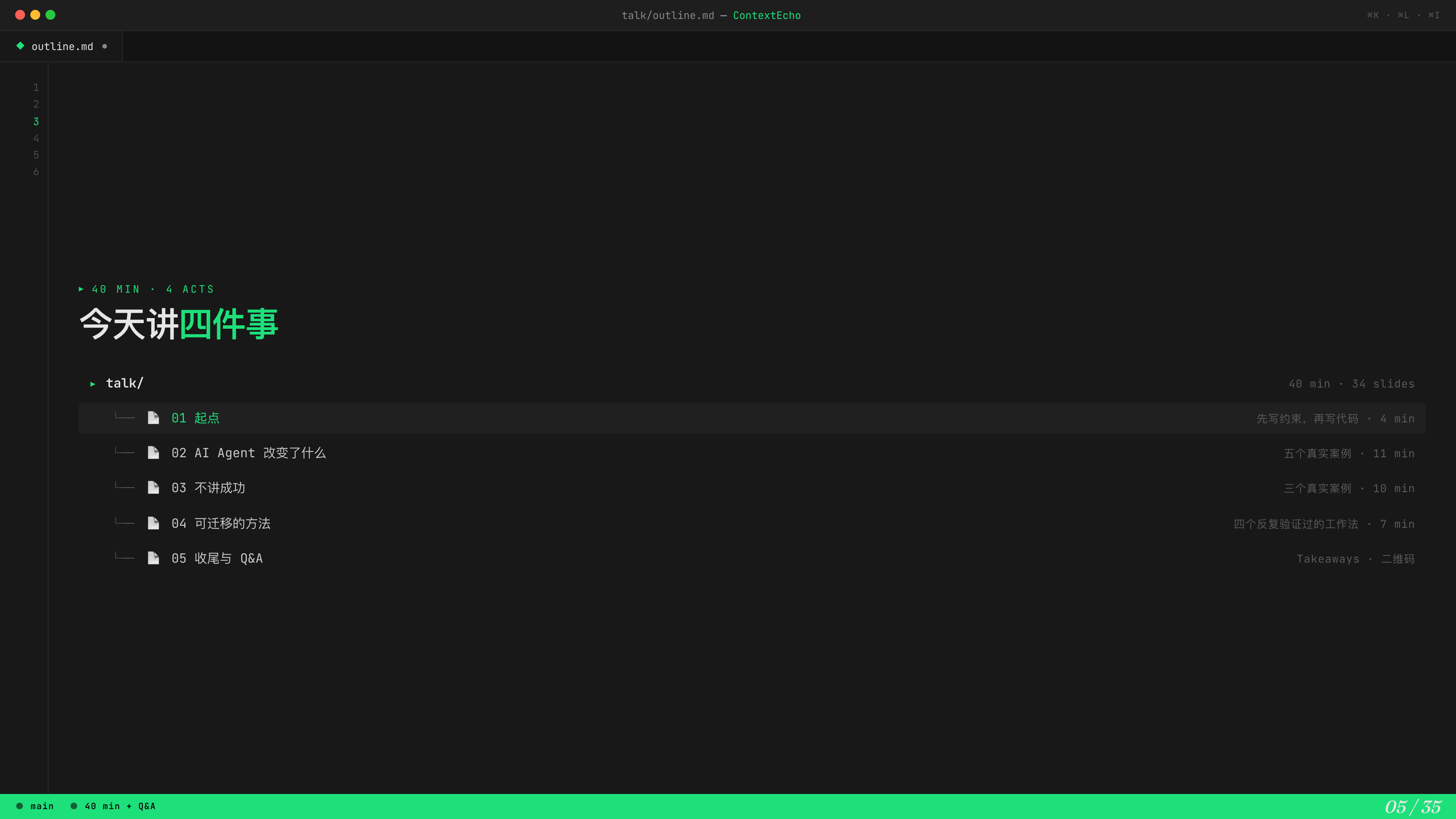
11条

Cursor rules + 7 篇 spec，每条背后是一次翻车。

最大一次砍功能

-4,830行

0.15.0 一个 PR，95 个文件；做产品也要敢于让 AI 做减法。



► 40 MIN · 4 ACTS

今天讲四件事

► talk/

40 min · 34 slides

- └─ 01 起点 先写约束，再写代码 · 4 min
- └─ 02 AI Agent 改变了什么 五个真实案例 · 11 min
- └─ 03 不讲成功 三个真实案例 · 10 min
- └─ 04 可迁移的方法 四个反复验证过的工作法 · 7 min
- └─ 05 收尾与 Q&A Takeaways · 二维码

—— 起点

先写约束， 再写代码

2026-04-14，第一条对话给的是 **Plan Mode**，不是「帮我写代码」。

01

ChatGPT

版本。因为是围绕具体内容进行的，而不是以电脑为主题来构建各种各样的输入内容和上下文内容，然后把这些汇集起来给 AI，而是围绕这个我在看的具体视频、音频、文章展开的一系列回顾。但是主要目的又不是单一的一个回顾。而是我把消费内容的载体就放在我自己构建的这个 app 上了。

这些内容来自于我自己先去收集的内容来导入。

展开

我先把你现在已经收敛出来的产品内核，整理成一套适合直接喂给 Cursor 的 MVP 文档骨架。然后我会给你一个“先做什么、不要做什么、首版页面和数据模型是什么”的版本，尽量让它能直接开工。

Thought for 1m 27s

可以，而且我建议你不要只给 Cursor 一份大 PRD。更适合的是给它一个很小但很硬的 MVP Build Pack。

你给的那个 PRD 模板核心结构是：Problem Statement、Solution、User Stories、Implementation Decisions、Testing Decisions、Out of Scope、Further Notes。 [GitHub](#)
这套结构很好，但对 Cursor 真正开工还不够。还需要把页面、交互、数据模型、技术边界补齐。

我建议你直接准备下面这 5 份文档，够开工，而且不会过重。

一、先定死 MVP 边界

先别让 Cursor 自己发散。



构建之前，先收敛需求

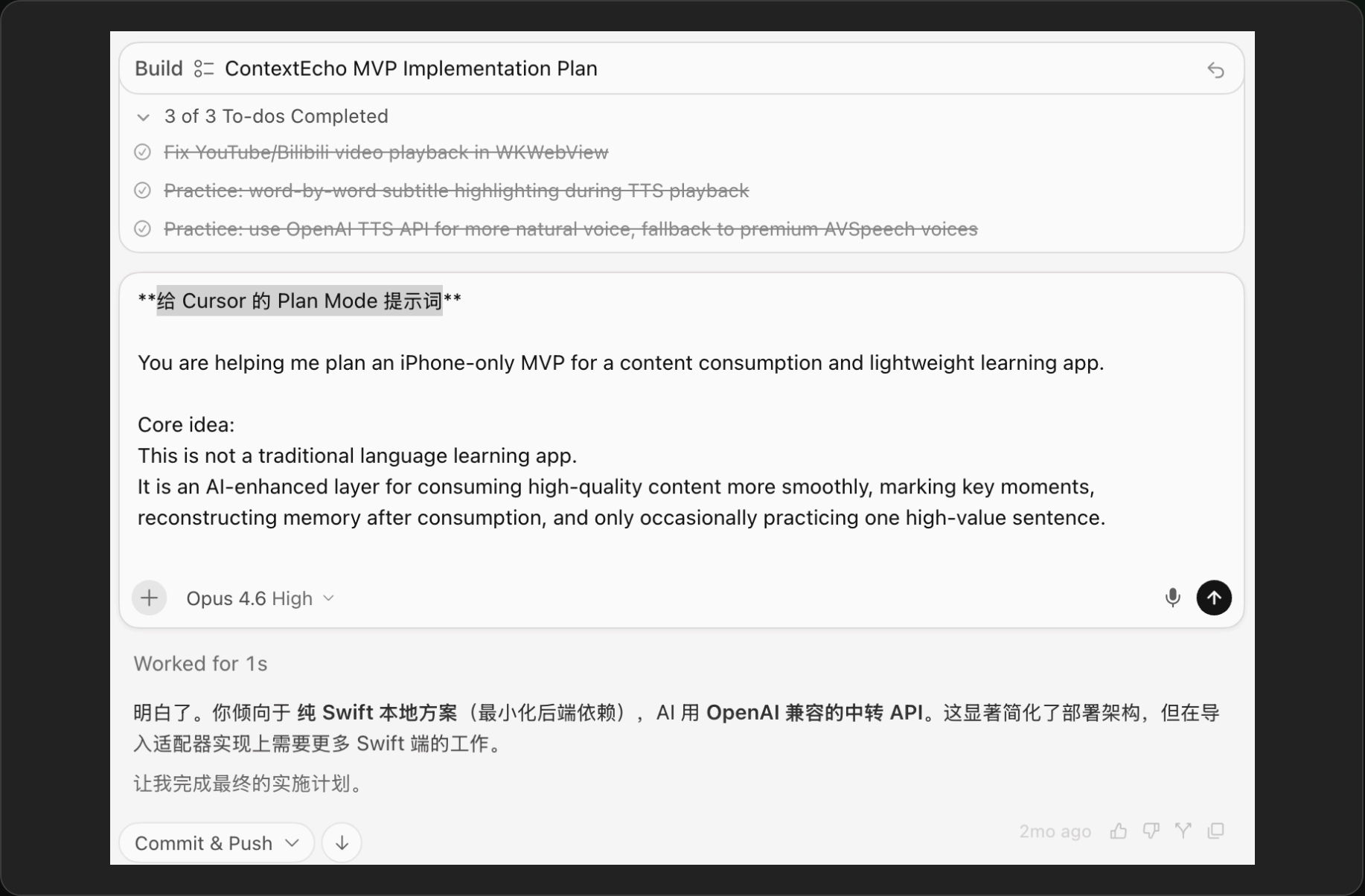
先把想法，收敛成可执行边界

我先和 ChatGPT 聊清需求。它没有直接给一份大 PRD，而是把想法整理成一个小而硬的 MVP Build Pack：做什么、不做什么、页面、交互与数据模型。

从 CHATGPT 到 CURSOR

拿到完整提示词， 才开始构建

This is **not** a traditional language learning app... 完整提示词带着产品定位、技术边界和明确的 To-dos 进入 Cursor Plan Mode。



Cursor · 完整提示词进入 Plan Mode，开始构建

1

2

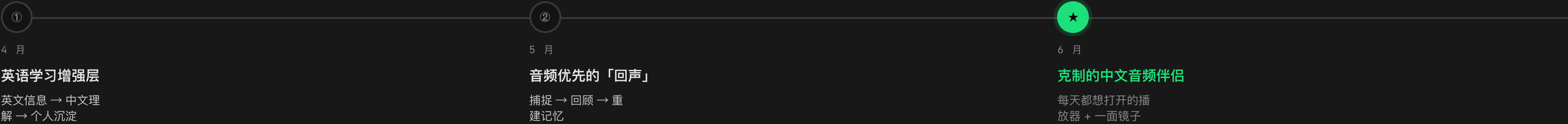
3

4

5

▸ 这个定义，后来被我推翻了两 次

伏笔：三次产品定位



AI AGENT 改变了什么

五个 真实案例

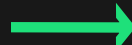
设计 · 调试 · 学习成本 · 架构 · 管理——五个具体塌掉的门槛，全部来自对话记录。

02

► 以前是流水线，现在是反馈环

以前 · 线性

- Idea → 设计 → 写代码 → 调试 → 发布
- 不会设计就卡在设计，不会调试就卡在调试
- 一个人做一款 iOS App 上架 ≈ 团队半年



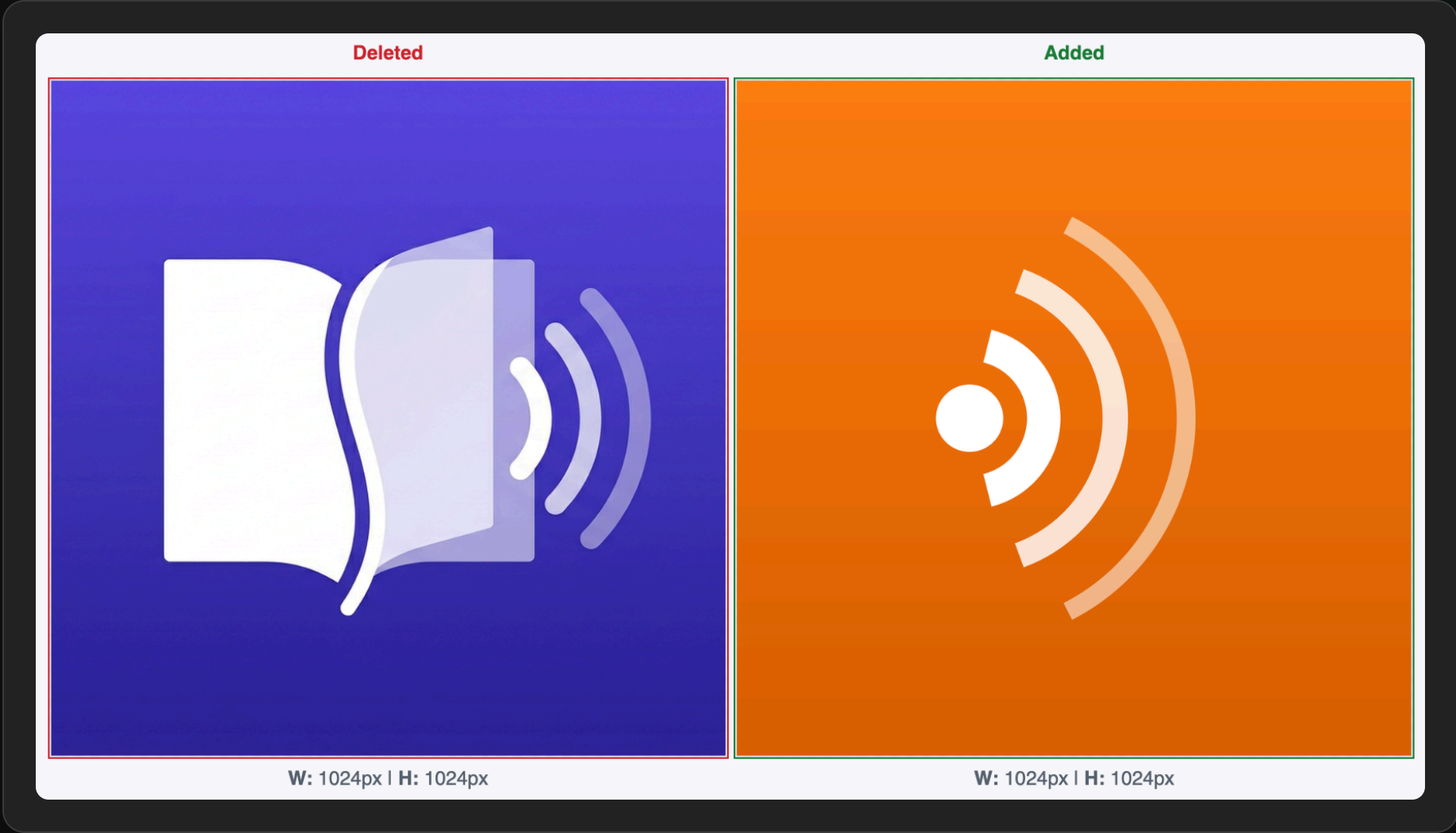
门槛塌了，循环转起来

- Idea $\rightleftharpoons$ AI $\rightarrow$ Prototype $\rightleftharpoons$ AI $\rightarrow$ 产品 $\rightleftharpoons$ 用户反馈 $\rightleftharpoons$ AI
- 卡住的地方，变成一次对话
- 瓶颈从工程能力，转移到判断力

案例① · 「我不喜欢紫色」

一次对话， 换掉全 App 的主题色

「我不喜欢紫色，很容易让人觉得 low，AI 味道很重。」—— 附一张 Overcast 截图 + 色值 #FC7E11，主题色、分类色、图标一次重构。



Before / After · 紫色 AI 味 → Overcast 橙

02-cases/debug-loop.md — diagnostics

debug-loop.md •

1

2

3

4

5

▶ 案例② • 不会 INSTRUMENTS 怎么办

逼 AI 给自己建可观测性

01

真机现象

「点击播客单集要卡 5 秒」，反复描述修不好。

02

转折点

「你看不到我手机——那就在 App 里建一套诊断日志导出。」

03

现场取证

跑步遇到闪退 → 导出日志 → 微信传回电脑。

04

数据回流

日志扔进对话，从「我描述现象」升级为「我提供数据」。

05

定位根因

SwiftData 主线程查询 + SwiftUI 每帧重绘 → 采样时钟 + 节流。

main

observability first

13 / 35

案例③ · 学习成本归零

贴一个失败的链接， AI 去研究协议

小宇宙、喜马拉雅、acast、Substack、Lex——每个内容源都是一套新方言。以前每个方言要读几天文档和逆向帖子，现在当天提问、当天上线。

01 **Bilibili**
WBI 签名 + 风控 -352 + dm_img 指纹对齐。

02 **YouTube**
embed error 152 / TLS 环境问题。

03 **Podcast RSS**
Podcast Index transcript 标准 + 各家时间轴格式。

案例④ · 对话原文

「增加的代码， 总比减少的多」

证据——播放一个领域堆出五个职责重叠的类：PlaybackBridge / PlaybackClock / PlaybackTimeBroker / PlayerSeekCoordinator / SmoothedClock。

ContextEcho 代码量 · 按 commit

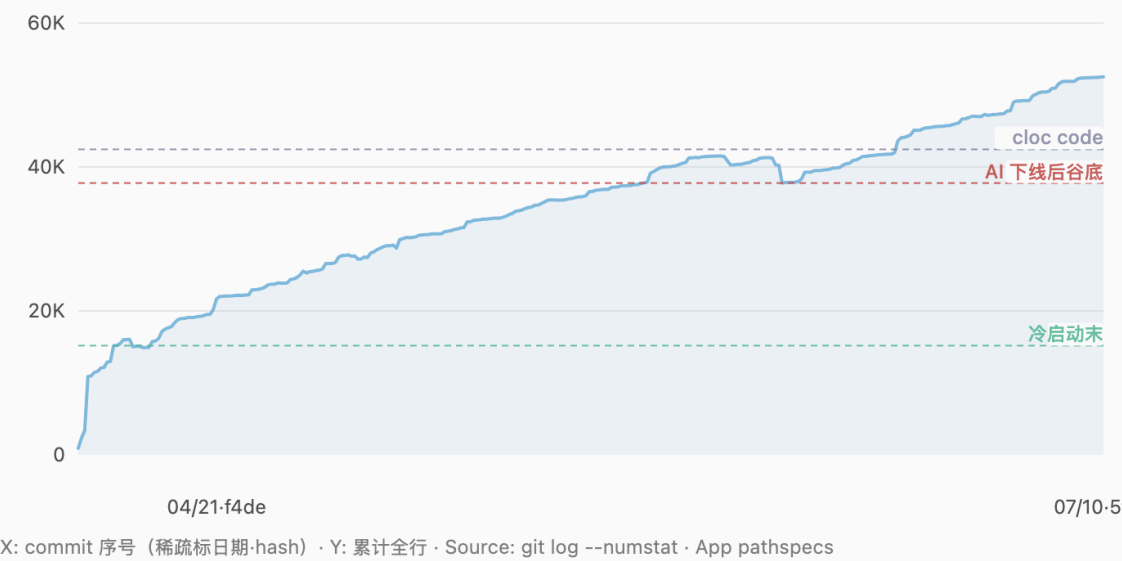
每个点 / 柱 = 一次触达 App Swift 的 commit (320 次) · 2026-04-16 → 2026-07-10 · 不做周聚合

42,453	320	+52,529	64
当前 code 行 (cloc)	App Swift commits	累计净增 (全行)	显著尖峰 (净 ≥300 或 churn≥1.5k)

为什么按 commit 而不是按周
周桶会把「单次 +7526 / 单次 -2488」摊进平均值。下面累计线是逐 commit 台阶；柱图只保留显著尖峰，并同时画写入(+)与删除(-)，重构会显示成一对高柱、净增接近零。

累计行数 · 逐 commit 台阶

横轴每个位置是一次 commit (按时间)。标签只标尖峰，避免 320 个字挤在一起；陡升 / 断崖就是单次大增删。



每个点都是一次 commit · 增长很快，收敛必须主动发生

.cursor/rules/ - ContextEcho

.cursor/rules/ •

1

2

3

4

5

6

▶ 案例⑤ • 每条规矩背后，都是一次真实翻车

给 AI 写「员工手册」

▶ .cursor/rules/

└─ 文件 xcodegen-workflow.mdc

└─ 文件 product-principles.mdc

└─ 文件 doc-sync-gate.mdc

└─ 文件 scope-boundary

└─ 文件 release-notes-sync.mdc

翻车事件 → 产出的规矩

新文件编译不过 • Cannot find X in scope

「你不太能遵循 iOS 开发基本理念」• 最高准则

文档与代码脱节 • 代码是事实，文档是意图

怕 AI 顺手加播放列表 • 「只支持单集播放」

发行说明漏写 • 每次 Minor 必须同步

main

11 rules

16 / 35

小结

以前我管代码，
现在我管一个不知疲倦、
但需要边界的合作者

—— 不 讲 成 功

三个 真实案例

被推翻四次的功能 · 六次演进的交互 · 连修三轮的翻车——**失败比成功更有信息量。**

03

1

2

3

4

5

▸ 案例 A · 产品名就来自这个功能

被推翻四次的「回声」



杀死「回声」的，不是 AI 的分析

是我自己 42天的使用数据

「我几乎不打开我们新设计出来的回声库.....目前已经连续 42 天，累计 97 小时 14 分钟。我可能是沉迷于用自己 build 的工具来收听。我不知道这个有何价值，但是我自己乐此不疲。」

Context Echo product positioning and functionality

我可以诚实的告诉你，我几乎不打开我们新设计出来的回声库。
近期也没有再怎么用过回声功能了。我每天都用它来听播客、看 bilibili 视频。对听过、看过的数据有一种成就感。目前已经连续 42 天，最长连续 42 天。累计 97 小时 14 分钟。

这段话非常诚实，谢谢你愿意把这层说出来。我先不着急给判断，我们一起把这个事实看清楚再说。

把你这段话翻译一下

你刚才其实回答了一个很重要的问题——你真实在用的是什么？

答案不是 vision 里写的那个"音频优先的捕获 + 回看 + 沉淀"闭环。

答案是：一个让你看到自己听了什么、听了多久、连续多少天的播客播放器。

具体说：

- 播放 = 真实在用
- 统计 / 连续天数 / 累计时长 / 每个节目的明细 = 真实在用，而且是给你愉悦感的主要来源
- 回声捕获 = 偶尔用（一周 3-5 次），打完不管
- 回声库 / 回看 / 沉淀 = 几乎不用
- AI 注解 / 单集回顾 = 我猜也几乎不用（你没提）

这意味着你 vision 里写的那个"主用户"——一个会回看、会沉淀、会形成知识资产的人——不是你。
而 vision 里降级到次要的"学习统计 / 连续天数 / 时长面板"——反而是你真实在用的核心。

你 vision 的主次关系，被你的真实行为彻底翻转了。

2026-05-28 · 真实使用行为推翻原产品定位

0.15.0 砍功能，还有一个新理由

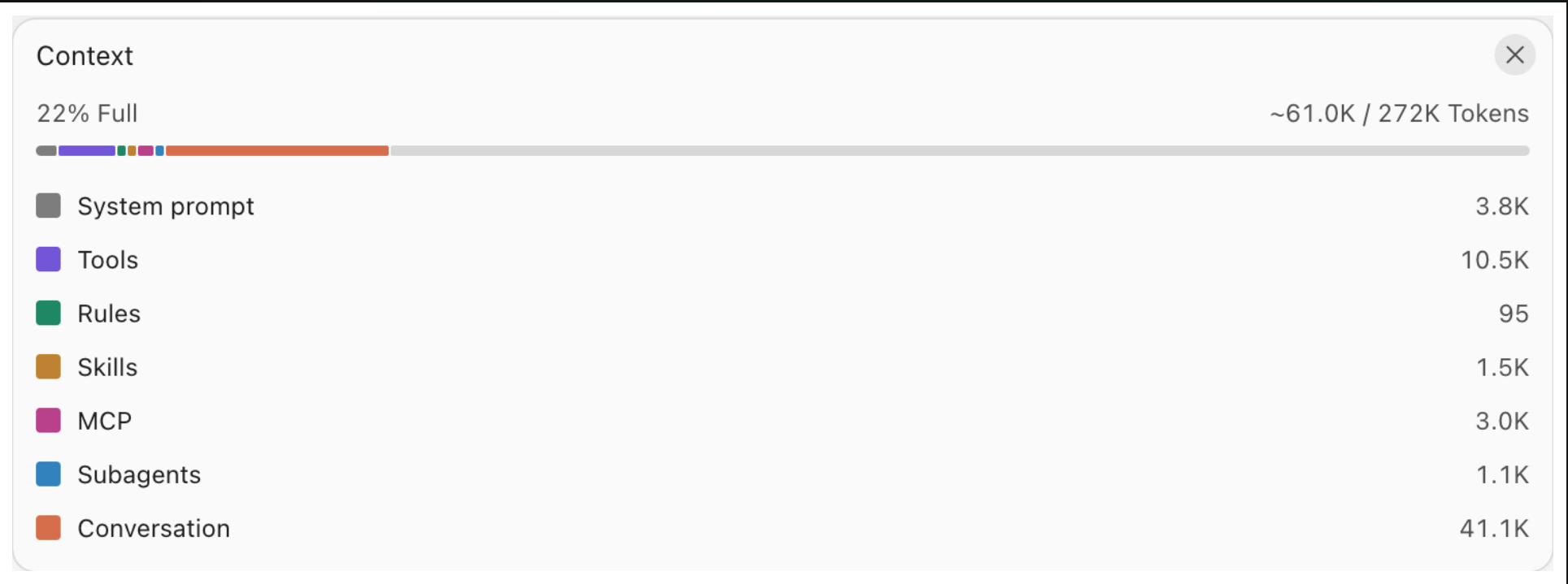
上下文有容量，更需要治理

- 01

容量不是可用空间
System prompt、Tools、Rules、Skills、MCP、Subagents 都会先占掉预算。
- 02

Conversation 通常最贵
对话越长，旧信息和无关任务越容易干扰当前判断。
- 03

治理就是主动做减法
一个任务一个对话；Plan 落盘；代码与文档对齐；砍功能也同步减 spec。



Context Window · 容量只是上限，结构决定 Agent 看见什么

1

2

3

4

5

▶ 案例 B · 每一步都来自跑道上的体感

一个交互的六次演进



案例 C · 对话原文

「你根本就没有修复到啊」

SwiftData + App Group 的一个坑，AI 连修三轮都是「改一改就算好了」。「.....你应该要认真一点，别乱改一改就算好了。」

「AI 搞不定创意和面向未来的产品方向，还是需要我自己来做分析、判断以及更深层次的决策。」

哪些决定， AI 给不了答案

01 创意与方向

从 0 到「做什么」是人的事——三次定位重写，输入全是「我自己怎么用」，AI 只负责结构化。

03 真实世界的体感

★ 该放左还是右、提示该停多久——AI 不跑步，也就给不出答案。

02 做减法的决心

AI 天然倾向做加法，从不主动建议删掉自己写的东西。

04 面对竞品的判断

Snipd 和我的想法惊人一致——是继续做还是被劝退，这个决定 AI 替不了。

可迁移的方法

我反复验证过的 四个工作法

不是抽象方法论，而是 76 天里反复救过我的固定动作，外加一张踩坑速查表。

04.

方法一

把判断力外化成制度

翻一次车，沉淀一条规则。检验标准——新开一个对话，AI 不需要你重复交代就能做对。

01 教 AI 写代码

xcodegen workflow、Swift 规范——最早的规则都是技术规范。

02 教 AI 懂产品边界

product principles、Scope Boundary——规则开始承载产品判断。

03 立流程闸门

doc gate / release-notes gate / version gate——制度自动把关。

方法二

上下文经济学

- 01 一个任务一个对话

混任务会让无关信息互相干扰，越聊越偏。
- 02 Plan 落盘，再新开对话执行

讨论产生的长对话别直接执行——落盘后的 plan 才是干净的意图。
- 03 代码是事实，文档是意图

两者对齐才是可用上下文；脱节的文档比没有更糟。
- 04 砍功能 = 减 spec = 减上下文税

给项目降复杂度，就是给 Agent 提理解精度。

方法三

逼 Agent 给自己建可观测性

01 · REPRODUCE

复现场景

稳定复现问题，收集第一手现象描述。



02 · BUILD

让 Agent 自建观测

没有专业工具？让它给自己造日志、埋点、导出通道。



03 · FEED

数据回流对话

日志、诊断文件、复现步骤——证据直接进上下文。



04 · FIX

对症下药

从「描述现象」升级为「提供数据」，一次定位根因。

方法四

把「不做什么」 写成硬规则

01 边界写进 rules

「暂时只支持单集播放」——防 AI 顺手补齐播放列表。

02 自动化要有确认层

嘉宾解析先建议、你确认才生效——自动化必须可撤销。

03 改边界先改规则

1.1.0 要做合集播放——先改 product-principles，再动代码。

76 天的坑，四类速查

踩坑速查表

SWIFTDATA

数据层

启动 crash · 主线程查询卡顿 · 迁移字段必须 optional。

SWIFTUI

性能

每帧重绘全页面 → 采样时钟 + 节流发布 · 巨型 View 拆分。

平台方言

内容源

RSS 各家一套时间轴 · B 站 WBI/-352 · YouTube 152/TLS。

AI 协作

所有人的坑

只加不减 · 乱改一改就算好 · 顺手扩大战场 · 上下文膨胀。

我没做的事

以前的「核心工作」

- 手写播放器
- 手调 RSS 解析
- 手改 UI 代码
- 手查 Instruments

我真正做的事

现在的核心竞争力

- 每天跑步用它——发现真问题
- 拍板砍回声库——用 42 天数据推翻自己
- 写下「只支持单集播放」——定义边界
- 决定用 Overcast 橙——审美与品味
- 带着困惑先上架——判断何时发布

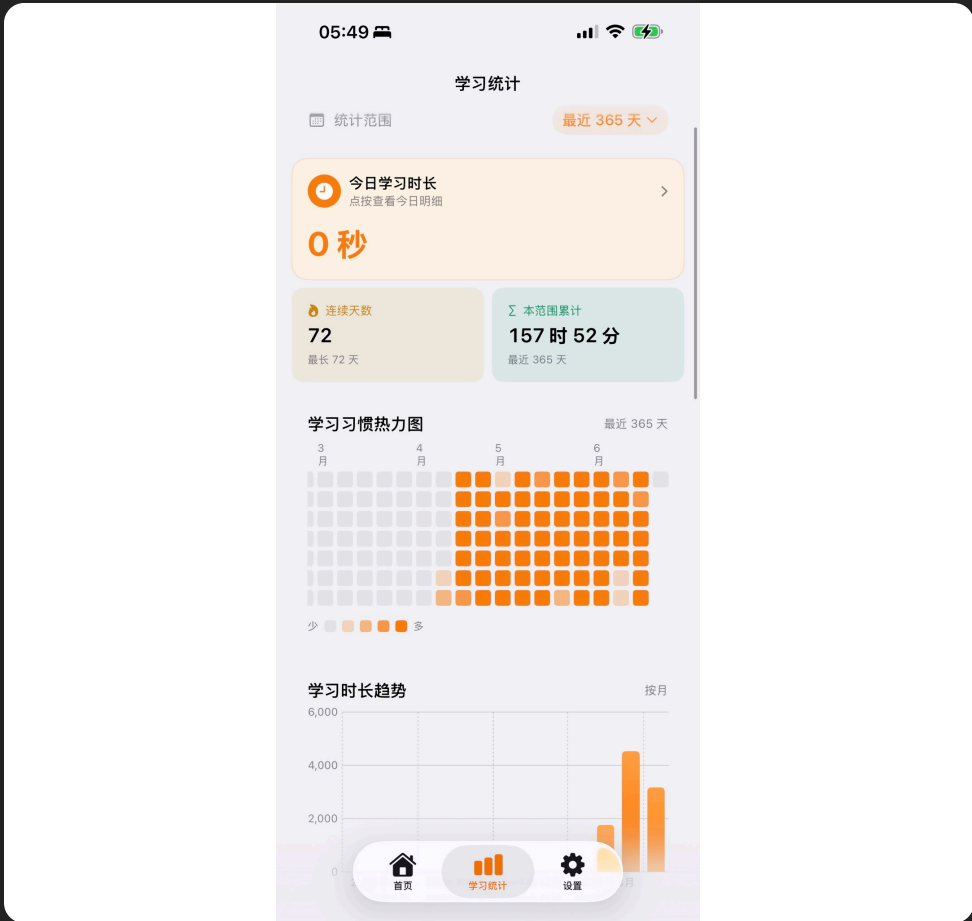
第 76 天，我没有等到完美答案

为了上架， 我先卸载了 自己用了72天的版本

买下苹果开发者账号，把只属于我的个人版换成公开发行版，然后从第 1 天重新开始。

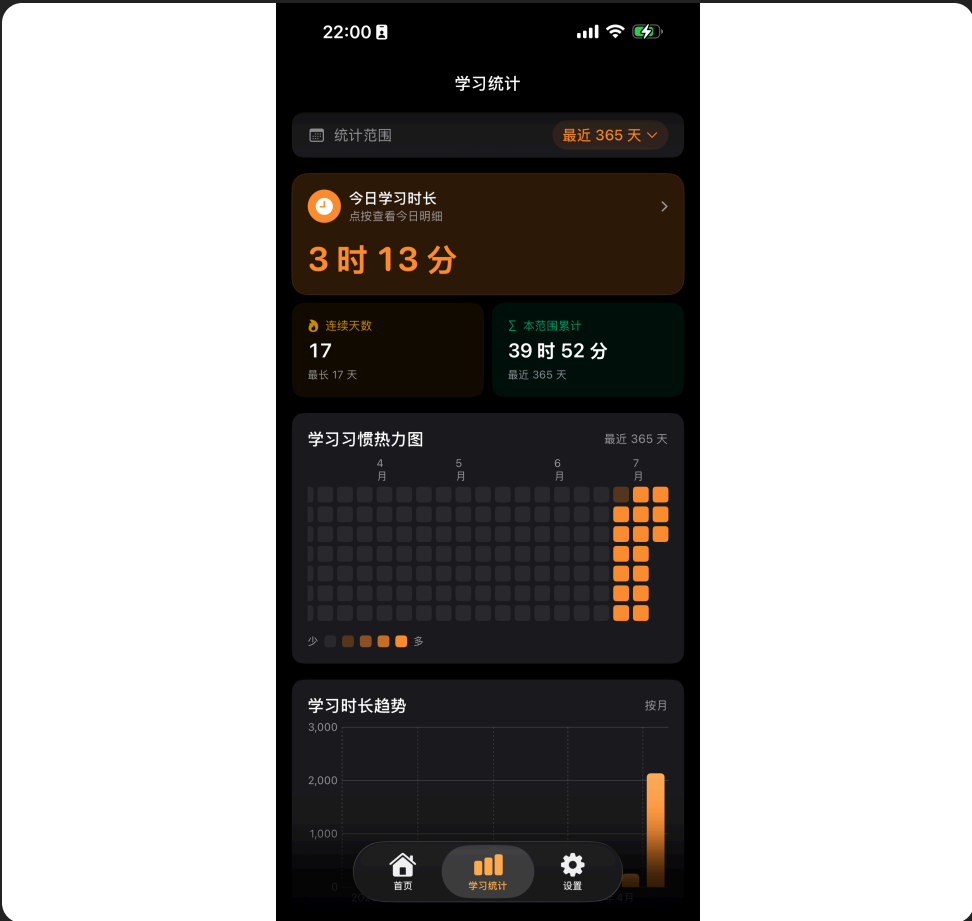
卸载前

PERSONAL BUILD · 72 DAYS



重新开始

APP STORE BUILD · 17 DAYS



正式上架后，我把这段故事发到 X：13K+ 浏览。公开发布不是终点，而是把自己的答案交给别人验证。

我的使用数据已经投了赞成票，现在轮到你

一款克制的 音频伴侣

ContextEcho 每天稳稳地播你想听的中英文播客和 B 站 / YouTube 合集；回到 App，你能看见自己在认真听什么——不刷推荐流，不抢注意力。

01 稳定续听

打开就接着上次的位置；Mini Player 常驻底部，一键回到正在听的那一集。

02 看见自己

连续天数、累计时长、最常听的节目、收听热力图——一套属于你的「自我画像」。

03 不打扰地陪着你

锁屏 / CarPlay / 耳机就绪，跑步、通勤、睡前都顺；不推送、不打卡。

```
everyone @ datafun : ~ $ start building █
```

别只围观 AI
还能做什么。
带上你最懂的问题，
现在就开始 Build。

下一个真正有价值的产品，也许就从今天在场的你手里开始。